



UNIVERSITI TEKNOLOGI MALAYSIA
FINAL EXAMINATION SEMESTER II, SESSION 2025/2026
(ALTERNATIVE ASSESSMENT)

COURSE CODE : SECR4483/SCSR4483
COURSE NAME : SECURE PROGRAMMING
SECTION : 01, 02, 03 & UTMSPACE
TIME : 9.00 AM (4 JULY 2026) – 9.00 AM (6 JULY 2026)
DATE/DAY : 4 – 6 JULY 2026 (TUESDAY – THURSDAY)
VENUES : UTM ELEARNING

Name	Muhamad Dzarhan bin Azmy
Matric No.	SX240663ECRHS04
Year / Course	2025/2026-II / SECRH
Section	UTMSPACE
Lecturer Name	DR. MOHD KUFAISAL BIN MOHD SIDIK

GitHub Link	
YouTube Link	

TABLE OF CONTENTS

CHAPTER 1	ARCHITECTURAL CODE REVIEW & VULNERABILITY DISCOURSE	5
1.1	Introduction	5
1.2	Vulnerability A & B/C search.php: SQL Injection and Reflected XSS	5
	Error! Bookmark not defined.	
1.2.1	Runtime Mechanism of the SQL Injection (Flaw A)	5
1.2.2	Runtime Mechanism of Reflected XSS (Flaws B & C)	5
1.3	Vulnerability D & E — auth.php: Bound Constraint Failure and Obsolete Cryptographic Primitive.	6
1.3.1	Byte-Length vs Character-Length Boundary (Flaw D)	6
1.3.2	Obsolete Cryptographic Primitive (Flaw E)	6
1.4	Vulnerability F & G — crypto_vault.php: ECB Mode and Hardcoded Key	7
1.4.1	Electronic Codebook (ECB) Pattern Leakage (Flaw F)	7
1.4.2	Hardcoded Cryptographic Key (Flaw G)	7
1.5	Secure Coding Principles Violated and PDPA 2010 Implications	8
CHAPTER 2	SECURE REFACTORING IMPLEMENTATION & EXPLOIT-CHAIN DISRUPTION MATRIX	11
2.1	Structural Dissection & Deconstruction of search.php	11
2.2	Boundary Characterisation & Cryptographic Profiling of auth.php	12
2.3	Exploit-Chain Disruption Matrix & Logical Defense	13
CHAPTER 3	CRYPTOGRAPHIC SUBSYSTEM ANALYSIS, PAYLOAD SERIALIZATION, & OPERATIONAL RESILIENCE	16

3.1	Legacy Mode Dissection: ECB vs AES-256-GCM AEAD	16
3.2	IV Dependency and Degradation Risk	16
3.3	Serialization Pipeline: Packing and Unpacking IV, Ciphertext and Tag	17
3.4	Interpreter Execution Flow: Authentication Tag Mismatch Handling	18
3.5	Environment Dependency Mapping	19
3.6	Automated Failure-Mode Diagnosis via PHPUnit	19
CHAPTER 4	SYSTEMIC VULNERABILITY CORRELATION, FORENSIC DOCUMENTATION, & ARCHITECTURAL SYNTHESIS	22
4.1	Forensic Exploit-Chain Dependency Analysis	22
4.2	Structural Synthesis and Cross-Module Correlation Table	22
4.2.1	r	Error! Bookmark not defined.
4.2.1.1	Sub-Sub-Sub Chapter	Error! Bookmark not defined.

LIST OF TABLE

Table 1. 1	Secure Coding Violations vs PDPA 2010 Obligations	9
Table 1. 2	<i>Before vs After — search.php (SQL Injection & Reflected XSS Remediation)</i>	12
Table 1. 3	Exploit Pathway Interruption Matrix	14
Table 4. 1	Structural Synthesis and Cross-Module Correlation Table	23

LIST OF FIGURE

Figure 3. 1 *Serialization Layout of the Encrypted Vault Payload* **Error! Bookmark not defined.**

CHAPTER 1

ARCHITECTURAL CODE REVIEW & VULNERABILITY DISCOURSE

1.1 Introduction

In this chapter we have done a thorough forensic code analysis of the three major software artifacts involved in the MediChain HealthVault-API security incident to examine search.php, auth.php and crypto_vault.php. Our goal is to deconstruct each flaw at the PHP/Zend runtime layer, explain how it was exploited and connect them back to the violated secure coding paradigms and their legal implications under the Malaysian Personal Data Protection Act (PDPA) 2010.

1.1.1 Runtime Mechanism of the SQL Injection (Flaw A)

The application automatically generates database commands by concatenating raw, unedited HTTP GET input into an active SQL string: "...WHERE name LIKE '%" . \$keyword . "%'"

At the PHP interpreter level, strings are handled as an opaque byte sequence and there is no difference between developer-authored commands and untrusted input. When `mysqli::query()` is called, this blended string is passed wholesale to MySQL server's SQL parser. Since the database parser tokenizes input sent through the wire protocol, an engineered payload like: `x%' UNION SELECT username, auth_key_hash, role FROM staff_credentials --`

effectively breaks the string literal boundary and introduces a completely new SQL query structure. The database engine cannot distinguish between legitimate application commands and malicious ones, as they are serialized in the same lexical stream before execution. This is typical of the disconnection of data plane and command plane, where data is assumed to be executable code. There is only a small code quality error where the variable is treated as an undefined bareword constant keyword and not \$ prefixed (no \$ prefix) and this would lead to an Error in modern PHP 8.x and stop execution. However, this analysis assumes that the syntax error is ignored by the developer to \$keyword.

1.1.2 Runtime Mechanism of Reflected XSS (Flaws B & C)

The application reads the unencoded \$keyword parameter back into the HTML response for both cases (regardless of whether database records are found) with `echo "...keyword: " . $keyword`. When the client browser reads this response, its internal HTML parser reads the entire stream as layout markup. The output is not encoded so HTML

metacharacters (<, >, ", ') are seen as structural and not just literal text. An injection string.

1.2 Vulnerability D & E — auth.php: Bound Constraint Failure and Obsolete Cryptographic Primitive.

1.2.1 Byte-Length vs Character-Length Boundary (Flaw D)

The defensive guard `if (strlen($inputKey) > 256)` evaluates the physical byte capacity of the string buffer rather than its semantic character length. Under the application's utf8mb4 encoding scheme, single multi-byte characters (such as emojis or complex ideographs) consume up to 4 bytes each. This structural discrepancy allows an attacker to construct an engineered payload that violates character-length business assumptions while slipping under the 256-byte constraint. Processing these crafted inputs forces the interpreter to allocate disproportionate CPU cycles toward string allocation, buffering, and hashing. Under high concurrent request volumes, this mismatch magnifies resource consumption, transforming a validation oversight into a highly effective Denial of Service (DoS) vector.

1.2.2 Obsolete Cryptographic Primitive (Flaw E)

User authentication relies on the evaluation `md5($inputKey) === $stored_hash`. Designed in 1991 exclusively for fast integrity checks, MD5 lacks salt mechanisms and computational hardness, making it completely unfit for password storage. Because it executes within minimal CPU cycles and has no work-factor adjustment, an offline attacker who extracts the `staff_credentials` hashes via the primary SQL injection vulnerability can execute billions of guess permutations per second using generic consumer GPUs. Furthermore, unsalted MD5 outputs are widely vulnerable to instant reversal using precomputed rainbow tables. While the strict equality operator `===` successfully protects against PHP type-juggling attacks (such as *magic hash* collisions), the underlying mathematical primitive remains entirely obsolete.

1.3 Vulnerability F & G — `crypto_vault.php`: ECB Mode and Hardcoded Key

1.3.1 Electronic Codebook (ECB) Pattern Leakage (Flaw F)

Executing `openssl_encrypt($medical_payload, 'aes-128-ecb', $secret_key)` partitions and encrypts data in standalone, isolated 16-byte blocks. Because there is no block chaining, feedback loops, or initialization vectors (IVs), identical blocks of plaintext processed under the same cryptographic key consistently map to completely identical ciphertext outputs. The system's sample data reveals this structural flaw: multiple patient files containing repeated strings (e.g., "DIAGNOSIS: Stage-2 Carcinoma. TREATMENT:") yield distinct, visible repeating ciphertext blocks. This structural leakage allows unauthorized observers to infer patient diagnostic categories and isolate therapeutic trends without decryption keys (*ECB penguin phenomenon*). Moreover, ECB provides no cryptographic integrity validation; an adversary can rearrange or delete individual ciphertext blocks without triggering decryption errors, facilitating covert medical data tampering.

1.3.2 Hardcoded Cryptographic Key (Flaw G)

The plaintext cryptographic token "MedVaultKey123!" is hardcoded directly into the application source as `$secret_key`. Storing secrets within version control exposes them to any entity with access to the source repository, server backups, compiled builds, or misconfigured web roots. The key cannot be rotated without a manual code redeployment, and its low entropy and human-readable format make it highly susceptible to offline dictionary guessing attacks.

1.4 Secure Coding Principles Violated and PDPA 2010 Implications

The table below maps each defect to the fundamental secure-coding principle it violates and the specific PDPA 2010 obligation placed at risk.

Defect	Secure Coding Principle Violated	PDPA 2010 Exposure
A — SQLi (search.php)	Separation of Data and Command Planes: user input must never be concatenated into an executable command string.	Security Principle (s.9): failure to take practical steps to protect personal data from unauthorised access/disclosure; enables the bulk exfiltration described in the incident chronology.
B/C — Reflected XSS (search.php)	Context-Aware Output Encoding: all output must be encoded for the syntactic context (HTML, JS, attribute) into which it is placed.	Security Principle (s.9) and Integrity Principle (s.11): enables session hijacking and unauthorised actions performed under a legitimate clinician's session.
D — Byte/Character Bound Mismatch (auth.php)	Input Validation Boundaries: validation must operate on the same semantic unit (characters, not raw bytes) that the business rule intends to constrain.	Security Principle (s.9): resource-exhaustion pathway degrades availability of a system processing sensitive health/authentication data.

Defect	Secure Coding Principle Violated	PDPA 2010 Exposure
E — MD5 Credential Hash (auth.php)	Cryptographic Primitive Agility / Adequate Credential Protection: use memory-hard, salted, work-factor-tunable hashing for credentials.	Security Principle (s.9): directly enabled the “Authentication Bypass Traversal” finding — reversed MD5 hashes produced valid, ephemeral “Physician” credentials used to approve unauthorised controlled-drug dispensations.
F — AES-ECB (crypto_vault.php)	Confidentiality and Integrity of Data at Rest: encryption mode must not leak plaintext structure and must be authenticated (AEAD).	Security Principle (s.9) and Integrity Principle (s.11): pattern leakage enabled patient re-identification by diagnosis cluster; lack of authentication enabled the described dosage-tampering/ransom stage.
G — Hardcoded Key (crypto_vault.php)	Secret Externalisation / Least-Privilege Key Custody: cryptographic material must live outside source control, in a managed secret store, with rotation support.	Security Principle (s.9): a single repository or artifact leak compromises confidentiality for the entire patient record set retroactively and prospectively.

Table 1. 1 Secure Coding Violations vs PDPA 2010 Obligations

The defects collectively show a systemic pattern, not just one or two bugs: at every trust boundary of the application - HTTP request into SQL, HTTP request into HTML, credential into storage, plaintext in cipher - the developers treated untrusted or sensitive material as though it were already safe for use in the next place. Under PDPA 2010, a Data User processing patient health data (a class of "sensitive personal data" under s.4) must take practical technical and organisational steps to mitigate the harm that comes from a breach; as mentioned above, the 72-hour deployment window that bypassed SAST and penetration testing is itself evidence relevant to a negligence finding, independent of any single code defect.

CHAPTER 2

SECURE REFACTORING IMPLEMENTATION & EXPLOIT-CHAIN DISRUPTION MATRIX

2.1 Structural Dissection & Deconstruction of search.php

Migrating to PDO parameterized queries introduces a structural boundary at the execution level. The `prepare()` function compiles the abstract query architecture on the database engine before any user input is accepted. The actual parameter values are later bound and transferred through a dedicated data channel using `execute()`. This ensures that user data can never alter the execution path or be interpreted as SQL commands. On the presentation layer, `htmlspecialchars()` maps the five critical HTML metacharacters into their respective inert string entities. This transformation guarantees that the browser processes the returned parameter strictly as safe display text, completely neutralizing executable markup.

VULNERABLE (BEFORE) — search.php	REFACTORED (AFTER) — search.php
<pre> <?php require_once 'C:\xampp\htdocs\Assesment\db_config.php'; \$keyword = \$_GET['keyword']; // Raw concatenation - data and command // planes are merged into one string \$sql = "SELECT id, name, illness_history " . "FROM patient_records " . "WHERE name LIKE '%" . \$keyword . "%'"; \$result = \$conn->query(\$sql); if (\$result->num_rows > 0) { while (\$row = \$result->fetch_assoc()) { // Unescaped reflection -> Reflected XSS echo "<div>Result found for keyword: " . \$keyword . "
"; echo "Patient: " . \$row['name'] . " History: " . \$row['illness_history'] . "</div><hr>"; } } else { echo "No records found for: " . \$keyword; } </pre>	<pre> <?php declare(strict_types=1); require_once __DIR__ . '/db_config.php'; // env-driven PDO factory \$keyword = \$_GET['keyword'] ?? ''; // 1. Data plane isolated from command plane: // the query text is fixed; only a bound // placeholder carries user data. \$stmt = \$pdo->prepare('SELECT id, name, illness_history FROM patient_records WHERE name LIKE :kw'); \$stmt->bindValue(':kw', '%' . \$keyword . '%', PDO::PARAM_STR); \$stmt->execute(); \$rows = \$stmt->fetchAll(PDO::FETCH_ASSOC); // 2. Context-aware output encoding on EVERY // variable placed into the HTML stream. \$safeKeyword = htmlspecialchars(\$keyword, ENT_QUOTES, 'UTF-8'); if (count(\$rows) > 0) { foreach (\$rows as \$row) { \$name = htmlspecialchars(\$row['name'], ENT_QUOTES, 'UTF-8'); </pre>

VULNERABLE (BEFORE) — search.php	REFACTORED (AFTER) — search.php
	<pre> \$hist = htmlspecialchars(\$row['illness_history'], ENT_QUOTES, 'UTF-8'); echo "<div>Result found for keyword: {\$safeKeyword}
"; echo "Patient: {\$name} History: {\$hist}</div><hr>"; } } else { echo "No records found for: {\$safeKeyword}"; } </pre>

Table 1. 2 *Before vs After — search.php (SQL Injection & Reflected XSS Remediation)*

2.2 Boundary Characterisation & Cryptographic Profiling of auth.php

Switching to `mb_strlen($str, 'UTF-8')` shifts the verification logic from counting raw bytes to evaluating explicit multi-byte characters. This ensures full alignment with application business rules.

To replace MD5, the application adopts the PASSWORD_ARGON2ID key derivation standard. Argon2id relies on strict memory allocation hurdles (`memory_cost`) and intensive CPU iterations (`time_cost`), preventing attackers from exploiting hardware acceleration structures (such as custom ASICs or mass GPU clusters) to break stolen hashes.

VULNERABLE (BEFORE) — auth.php	REFACTORED (AFTER) — auth.php
<pre> <?php if (\$_SERVER['REQUEST_METHOD'] === 'POST') { \$inputKey = \$_POST['auth_key']; // Byte-length check, not character-length if (strlen(\$inputKey) > 256) { die("Fatal Error: Bound overflow detected."); } // Fast, unsalted, non-memory-hard digest \$stored_hash = "098f6bcd4621d373cade4e832627b4f6"; if (md5(\$inputKey) === \$stored_hash) { echo "Access Granted."; } } </pre>	<pre> <?php declare(strict_types=1); if (\$_SERVER['REQUEST_METHOD'] === 'POST') { \$inputKey = \$_POST['auth_key'] ?? ''; // Semantic, charset-aware boundary check: // counts CHARACTERS, immune to multi-byte // byte-inflation bypass. if (mb_strlen(\$inputKey, 'UTF-8') > 256) { http_response_code(400); die('Invalid credential length.');</pre> <pre> } // Argon2id: memory-hard + time-hard KDF. // Hash fetched by username, never hardcoded. \$username = \$_POST['username'] ?? ''; \$stmt = \$pdo->prepare('SELECT auth_key_hash FROM staff_credentials WHERE username = :u'); \$stmt->bindValue(':u', \$username, PDO::PARAM_STR); \$stmt->execute(); \$row = \$stmt->fetch(PDO::FETCH_ASSOC); </pre>

VULNERABLE (BEFORE) — auth.php	REFACTORED (AFTER) — auth.php
	<pre> // Always run verify (constant-time-shaped // control flow) to resist username enumeration. \$hash = \$row['auth_key_hash'] ?? '\$argon2id\$dummy'; if (\$row && password_verify(\$inputKey, \$hash)) { echo 'Access Granted.'; } else { http_response_code(401); echo 'Access Denied.'; } } // One-time migration helper for issuing new hashes: // password_hash(\$key, PASSWORD_ARGON2ID, [// 'memory_cost' => 65536, 'time_cost' => 4, // 'threads' => 2]); </pre>

Table 2.2: Before vs After — auth.php (Bound Constraint & Credential Hashing Remediation)

2.3 Exploit-Chain Disruption Matrix & Logical Defense

The matrix below traces each original exploit pathway against the specific structural change that interrupts it, and explains why the interruption is mechanistic (i.e., holds for any payload shape) rather than a signature match against known-bad strings.

Exploit Pathway (Before)	Structural Break (After)	Why It Is Mechanistic, Not Signature-Based
Dynamic parameters are injected to append UNION SELECT extraction clauses.	PDO prepare()/bindValue() separates query skeleton compilation from value binding.	Input variables never interface with the database engine's parser; no character payload can alter the precompiled abstract syntax tree of the statement.
<script> tags are reflected to hijack	htmlspecialchars(ENT_QUOTES) systematically escapes the five critical HTML control characters.	Encoding operates on the character set itself (<, >, &, ", ') rather

Exploit Pathway (Before)	Structural Break (After)	Why It Is Mechanistic, Not Signature-Based
cookies or drop malicious forms.		than on known malicious substrings, so novel payloads, obfuscation, and encoding tricks are neutralised identically to well-known ones.
Multi-byte payload inflates byte length past intended character ceiling, degrading availability under concurrency.	<code>mb_strlen('UTF-8')</code> counts decoded logical characters, matching the intended business rule.	The check now measures the same unit the rule is written in; no byte/character unit mismatch remains for any encoding-crafted payload to exploit.
Compromised database hashes are reversed offline via high-speed MD5 cracking.	Authentication upgrades to Argon2id key derivation using <code>password_verify()</code> .	The memory-hard and time-hard parameters require fixed hardware allocations for every guess, making mass parallel cracking economically and computationally impossible.

Table 1. 3 Exploit Pathway Interruption Matrix

Taken together, the refactor breaks the exploit chain at two independent trust boundaries (data→SQL and data→HTML) in `search.php`, and at two independent control points (validation boundary and credential-storage primitive) in `auth.php`. Since SQLi was shown in Chapter 1 to be the pivot that exposed the MD5 hash column in the first place, closing the SQLi vector also removes the practical delivery path for the credential-reversal stage of the incident, even before the Argon2id migration is considered in isolation.

CHAPTER 3

CRYPTOGRAPHIC SUBSYSTEM ANALYSIS, PAYLOAD SERIALIZATION, & OPERATIONAL RESILIENCE

3.1 Legacy Mode Dissection: ECB vs AES-256-GCM AEAD

The refactored subsystem adopts AES-256-GCM, moving away from ECB to run a streaming counter mechanism for encryption. This ensures that identical blocks of plaintext map to completely unique cipher text fragments. Concurrently, GCM processes data using a GHASH polynomial loop to yield a 16-byte Authentication Tag. This provides authenticated encryption (AEAD); any bit-level modification of the ciphertext causes verification to fail during decryption, blocking unauthorized data injection or block swapping.

3.2 IV Dependency and Degradation Risk

GCM's security proof assumes the (key, IV) pair is never reused. Internally, the IV seeds the initial counter block fed into the block cipher to generate the keystream; if the same 96-bit IV is reused under the same key for two different plaintexts, then the two keystreams are identical and thus XORing the two ciphertexts cancels the keystream and gives the XOR of the two plaintexts to GCM. Worse, GCM's authentication is also undermined by IV reuse; if an attacker can see two messages encrypted under the same (key, IV), he/she can algebraically recover the GHASH authentication key and then can create fake authentication tags for any ciphertext. This is why the new code generates a new IV from a random source of random bytes (`random_bytes(12)`) for every single encryption call instead of a counter or timestamp or any other predictable state for that matter, and therefore the IV states in predictable states destroy the very randomness that the AEAD security proof is based on.

3.3 Serialization Pipeline: Packing and Unpacking IV, Ciphertext and Tag

At encryption time, `openssl_encrypt()` with `OPENSSL_RAW_DATA` and the `aes-256-gcm` cipher only returns the raw ciphertext bytes, and the 16-byte tag is written into a variable passed by reference and must be explicitly recorded by the code or it will be lost. The new `vault_encrypt()` function then combines the three logically distinct byte segments into one transportable blob in a fixed order: the 12-byte IV, the 16-byte Authentication Tag, and the variable-length ciphertext, and base64-encodes the result to be transferred over JSON/HTTP. At decryption time, `vault_decrypt()` cuts the base64-decoded blob at a fixed position (bytes [0:12) for the IV, [12:28) for the tag, and [28:end) for the ciphertext) and calls `openssl_decrypt()` with the tag provided. Failure to accurately encode the triple leads to a length-guard error ("Malformed payload") or a tag-verification failure, but the error is not allowed to propagate as an unhandled fatal error.

Refactored code:

```
<?php
// crypto_vault.php - Secure Refactor (AES-256-GCM)
require_once __DIR__ . '/vendor/autoload.php';
use Dotenv\Dotenv;
Dotenv::createImmutable(__DIR__)->load();

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $medical_payload = $_POST['payload'] ?? '';

    // Secret now lives outside the codebase entirely
    $secret_key = base64_decode($_ENV['VAULT_KEY_B64']); // 32 raw bytes for AES-256

    $iv = random_bytes(12); // fresh, unpredictable nonce every call
    $tag = '';

    $ciphertext = openssl_encrypt(
        $medical_payload, 'aes-256-gcm', $secret_key,
        OPENSSL_RAW_DATA, $iv, $tag, '', 16
    );

    if ($ciphertext === false) {
        http_response_code(500);
    }
}
```

```

        echo json_encode(['status' => 'error', 'message' => 'Encryption failed.']);
        exit;
    }

    // Pack IV(12) + Tag(16) + Ciphertext, then base64 for transport
    $packed = base64_encode($iv . $tag . $ciphertext);
    echo json_encode(['status' => 'vaulted', 'data' => $packed]);
}

function vaultDecrypt(string $packedB64, string $key): string {
    $raw = base64_decode($packedB64);
    $iv      = substr($raw, 0, 12);
    $tag      = substr($raw, 12, 16);
    $ciphertext = substr($raw, 28);

    $plaintext = openssl_decrypt(
        $ciphertext, 'aes-256-gcm', $key,
        OPENSSL_RAW_DATA, $iv, $tag
    );

    if ($plaintext === false) {
        // Tag mismatch -> tampering detected. Fail closed, isolated exception,
        // never a raw fatal crash that could leak a stack trace.
        throw new RuntimeException('AEAD authentication failed: payload rejected.');
```

3.4 Interpreter Execution Flow: Authentication Tag Mismatch Handling

If a payload is modified or an authentication tag mismatch occurs, `openssl_decrypt()` returns a boolean false value. The refactored module intercepts this failure within a structured try-catch block, throwing a clean `RuntimeException` that maps to an HTTP 500 JSON error. Handling this within application logic prevents unhandled PHP fatal errors or runtime memory crashes, neutralizing potential availability (DoS) attacks.

3.5 Environment Dependency Mapping

The refactored key can be read via `getenv('VAULT_AES_KEY')` from a `.env` file loaded by a `dotenv`-style loader at bootstrap, removing the secret from the PHP source tree entirely. Thus the dependency graph is application code → environment loader → `.env` file (that is present only on the deployment host, not on the web root) → OS-level file permissions that limit how the application service can read it. A corresponding `.gitignore` entry (`.env`, `*.env.local`) structurally removes the file from what git tracks or transmits to any remote repository, so a leak of the repository from a public fork, a CI artifact, an accidental push will not expose the key, and the attack surface is reduced from "anyone who clones the repo" to "anyone who has OS-level or secret-manager-level access to the running host."

Cryptographic keys are completely separated from the codebase by retrieving them from runtime environment variables:

```
getenv('VAULT_AES_KEY')
```

The `.env` configuration file is restricted using native operating system permissions and excluded from version control via `.gitignore`, preventing accidental exposures on shared repositories.

3.6 Automated Failure-Mode Diagnosis via PHPUnit

A comprehensive automated test suite (PHPUnit) is implemented to validate three distinct operational states:

1. **Positive Path Execution:** Asserts that data passing through the encryption and decryption pipeline remains identical to the source input via `assertSame()`.
2. **Integritic Mismatch Validation:** Simulates data tampering by mutating a byte within the ciphertext array, verifying that the system catches the error and throws a `RuntimeException` using `expectException()`.
3. **Credential Integrity Verification:** Validates that `password_verify()` accurately passes correct credentials and rejects invalid inputs under the Argon2id standard.

```

<?php
use PHPUnit\Framework\TestCase;

final class CryptoVaultTest extends TestCase
{
    private string $key;

    protected function setUp(): void {
        $this->key = random_bytes(32); // 256-bit test key
    }

    public function testEncryptDecryptRoundTrip(): void {
        $plaintext = 'DIAGNOSIS: Stage-2 Carcinoma.';
        $iv = random_bytes(12); $tag = '';
        $ct = openssl_encrypt($plaintext, 'aes-256-gcm', $this->key,
            OPENSSL_RAW_DATA, $iv, $tag, '', 16);
        $pt = openssl_decrypt($ct, 'aes-256-gcm', $this->key,
            OPENSSL_RAW_DATA, $iv, $tag);
        $this->assertSame($plaintext, $pt);
    }

    public function testTamperedCiphertextThrowsOnDecrypt(): void {
        $plaintext = 'TREATMENT: Insulin regimen.';
        $iv = random_bytes(12); $tag = '';
        $ct = openssl_encrypt($plaintext, 'aes-256-gcm', $this->key,
            OPENSSL_RAW_DATA, $iv, $tag, '', 16);
        $tamperedCt = $ct;
        $tamperedCt[0] = $tamperedCt[0] === "\x00" ? "\x01" : "\x00";

        $result = openssl_decrypt($tamperedCt, 'aes-256-gcm', $this->key,
            OPENSSL_RAW_DATA, $iv, $tag);
        $this->assertFalse($result, 'AEAD must reject tampered ciphertext.');
```

```

    }

    public function testCredentialHashIntegrity(): void {
        $plainKey = 'doctorsecret';
        $hash = password_hash($plainKey, PASSWORD_ARGON2ID);
        $this->assertTrue(password_verify($plainKey, $hash));
        $this->assertFalse(password_verify('wrongkey', $hash));
    }
}

```

}

CHAPTER 4

SYSTEMIC VULNERABILITY CORRELATION, FORENSIC DOCUMENTATION, & ARCHITECTURAL SYNTHESIS

4.1 Forensic Exploit-Chain Dependency Analysis

The architectural assessment demonstrates that the application's defects do not exist in isolation; they form an interconnected exploit chain where an upstream validation vulnerability cascades into downstream system compromises:

- Stage 1 (Initial Access): The lack of query parameterization in search.php exposes a foundational SQL injection flaw.
- Stage 2 (Privilege Escalation): Because the database connection profile has over-privileged system rights, the attacker can move laterally beyond the patient record space to extract the staff_credentials data cache.
- Stage 3 (Credential Reversal): The use of unsalted MD5 hashing allows the attacker to quickly reverse stolen credential signatures offline, gaining administrative authentication rights within auth.php.
- Stage 4 (Data Manipulation): Weak at-rest encryption (ECB mode and hardcoded keys) removes the final line of defense, letting the attacker alter sensitive medical treatment values without detection.

4.2 Structural Synthesis and Cross-Module Correlation Table

Stage	Originating Module	Mechanism	Downstream Consequence
Data Access	search.php	Unparameterised SQL concatenation over a root-privileged DB connection	Full schema enumeration, including tables unrelated to patient search

Stage	Originating Module	Mechanism	Downstream Consequence
Data Access	search.php → auth.php	UNION-based extraction of staff_credentials.auth_key_hash	Exposure of the entire legacy MD5 credential store
Auth check	auth.php	Offline reversal of unsalted, non-memory-hard MD5 hashes	Forged/derived valid credentials used to approve controlled-substance dispensation
Data at rest	crypto_vault.php	ECB pattern leakage + hardcoded, unrotatable AES key	Patient re-identification by diagnosis cluster and undetectable dosage tampering

Table 4. 1 Structural Synthesis and Cross-Module Correlation Table

None of the fixes used here are anything special or new, parameterised queries, output encoding, Argon2id, and AES-GCM are all things that have existed in PHP for years and are honestly considered the normal way to do things now (PDO and password_hash() have been part of core PHP for a long time). MediChain got breached simply because the 72-hour deadline meant these normal practices got skipped, not because the bugs were some kind of advanced zero-day. I think that's really the lesson from this whole case study, skipping SAST and pentest doesn't make the vulnerabilities disappear, it just decides who finds them first, the dev team or the attacker.

REFERENCES

- Malaysia. (2010). Personal Data Protection Act 2010 (Act 709). Attorney General's Chambers of Malaysia.
- OWASP Foundation. (2021). OWASP Top Ten 2021: A03:2021 – Injection. <https://owasp.org/Top10/>
- OWASP Foundation. (2021). OWASP Top Ten 2021: A02:2021 – Cryptographic Failures. <https://owasp.org/Top10/>
- PHP Documentation Group. (n.d.). PDO — PHP Data Objects. <https://www.php.net/manual/en/book.pdo.php>
- PHP Documentation Group. (n.d.). password_hash — PASSWORD_ARGON2ID. <https://www.php.net/manual/en/function.password-hash.php>
- PHP Documentation Group. (n.d.). openssl_encrypt / openssl_decrypt. <https://www.php.net/manual/en/function.openssl-encrypt.php>
- NIST. (2007). Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC (SP 800-38D). National Institute of Standards and Technology.
- Biryukov, A., Dinu, D., & Khovratovich, D. (2016). Argon2: The memory-hard function for password hashing and other applications. IACR.